EXPERIMENT NO.1

Study Basics of OpenMP API (Open Multi-Processor API)

Title: Study Basics of OpenMP API (Open Multi-Processor API)

Objectives: To study Study Basics of OpenMP API (Open Multi-Processor API)

.

Aim/ Problem Statement: To study Study Basics of OpenMP API (Open Multi-Processor API)

Theory:

What is OpenMP?

OpenMP (Open MultiProcessing) is a parallel programming model based on compiler directives which allows application developers to incrementally add parallelism to their application codes.

OpenMP API specification for parallel programming provides an application programming interface (API) that supports multi-platform shared memory multiprocessing programming in C, C++, and Fortran, on most platforms. It consists of a set of compiler directives, library routines, and environment variables that influence run-time behavior.

Since OpenMP focuses on the parallelism within a node (shared memory multiprocessing) it can be combined with message-passing programming models, such as MPI, to execute on multiple nodes.

Compilation#

There are many compilers that support different versions of the OpenMP specification. OpenMP maintains a list here with the compiler that supports it and the supported version. In general, to compile (and link) an application with OpenMP support you need only to add a compile flag and if you use the OpenMP API you need to include the OpenMP header (omp.h). While the header file has a fixed name, the compile flag depends on the compiler. The following is a non-exhaustive list of compilers and the flag that enables OpenMP.

- GCC (including gcc, g++ and gfortran) : -fopenmp
- LLVM: -fopenmp

- Intel compiler-suite (including icc, icpc and ifort) : `-qopenmp` (and `-fopenmp` for compatibility with GCC/LLVM)
- IBM XL compiler-suite (including xlc, xlC and xlf) : `-xlsmp=omp`
- PGI compiler-suite (including pgcc pgc++ pgfortran) : '-mp'

Parallel hello world using OpenMP#

```
#include <omp.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
#pragma omp parallel
{
printf ("Hello world! I'm thread %d out of %d threads.\n",
omp_get_thread_num(), omp_get_num_threads());
}
return 0;
}
```

This code simply creates a team of threads (according to the environment variable `OMP_NUM_THREADS` - and if not defined will create one per logical core on the system) and each thread will identify itself besides printing the typical Hello world message.

}

OpenMP parallel region construct:

•Block of code to be executed by multiple threads in parallel

•Each thread executes the same code redundantly (SPMD)

•Work within work-sharing constructs is distributed among the threads in a team

•Example with C/C++ syntax

#pragma omp parallel [clause [clause] ...] new-line structured-block

•clause can include the following:

private (list)

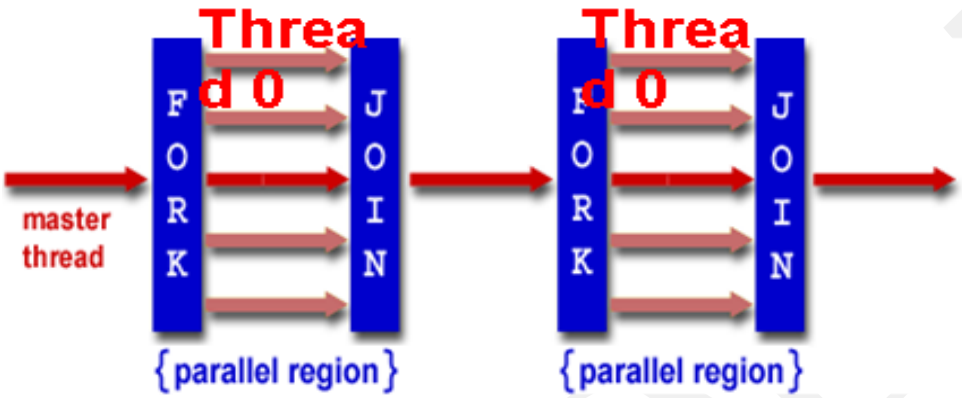
shared (list)

•Example: OpenMP default is shared variables To make private, need to declare with pragma:

#pragma omp parallel private (x)

OpenMP Programming Model - Review

- **Fork - Join Model:**



OpenMP programs begin as single process (*master thread*) andexecutes sequentially until the first parallel region construct is encountered

FORK: Master thread then creates a team of parallel threads

Statements in program that are enclosed by the parallel region construct areexecuted in parallel among the various threads

JOIN: When the team threads complete the statements in the parallel regionconstruct, they synchronize and terminate, leaving only the master thread

CONCLUSION:

We have studied the all Basics of OpenMP API (Open Multi-Processor API).

FAQs:

1. What is OpenMPI?
2. Which compilers support OpenMP?
3. What problem does OpenMP solve ?
4. What is fork-join model in OpenMPI?